Axion – EV-Fleet -Management

MINOR PROJECT REPORT

Submitted in partial fulfillment of the requirement for the Degree of Bachelors of Engineering in Computer Science & Engineering

Submitted To:

[PARUL UNIVERSITY, VADODARA, GUJARAT (INDIA)]

Submitted By:

Animesh Sharma (2303051050085)
Kajol Gehlot (2303051051204)

Under The Guidance of:

Mr. Meet Kumar Patel
(Assistant Professor, CSE)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PARUL INSTITUTE OF TECHNOLOGY VADODARA, GUJARAT

SESSION: AY 2025-2026

Parul University
Parul Institute of Technology

(Session: 2025 -2026)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that **Animesh Sharma & Kajol Gehlot**, Students of CSE **VI Semester** of **" Parul Institute of Technology, Vadodara"** has completed their Minor Project titled **"Axion – EV-Fleet Management",** as per the syllabus and has submitted a satisfactory report on this project as a partial fulfillment towards the award of degree of Bachelor of Technology in Computer Science and Engineering under Parul University, Vadodara, Gujarat (India).

Mr.Meet Kumar Patel	**Asst.Prof.SumitrMenaria**
(Project Guide)	**Head (CSE)**
(Assistant Professor)	**PIT, Vadodara**
(CSE)	

DECLARATION

We the undersigned solemnly declare that the project report "**Axion – EV-Fleet-Management"** is based on my own work carried out during the course of our study under the supervision of **Mr. Meet Kumar Patel, Assistant Professor, CSE**.

We assert the statements made and conclusions drawn are the outcomes of my own work. I further certify that

1. The work contained in the report is original and has been done by us under the general supervision of our supervisor.
2. The work has not been submitted to any other Institution for any other degree / diploma / certificate in this university or any other University of India or abroad.
3. We have followed the guidelines provided by the university in writing the report.

Whenever we have used materials (data, theoretical analysis, and text) from other sources, we have given due credit to them in the text of the report and giving their details in the references.

Animesh Sharma [2303051050085]

Kajol Gehlot [2303051051204]

ACKNOWLEDGEMENT

In this semester, we have completed our project on **"Axion – EV-Fleet-Management"**. During this time, all the group members collaboratively worked on the project and learnt about the industry standards that how projects are being developed in IT Companies. We also understood the importance of teamwork while creating a project and got to learn the new technologies on which we are going to work in near future.

We gratefully acknowledge for the assistance, cooperation guidance and clarification provided by **Mr. Meet Kumar Patel** during the development of our project. We would also like to thank our Head of Department **Prof. Sumitra Menaria** and our Principal **Dr. Swapnil Parikh** Sir for giving us an opportunity to develop this project. Their continuous motivation and guidance helped us overcome the different obstacles for completing the Project.

We perceive this as an opportunity and a big milestone in our career development. We will strive to use gained skills and knowledge in our best possible way and we will work to improve them.

Animesh Sharma [2303051050085]

Kajol Gehlot [2303051051204]

LIST OF FIGURES

S. No.	Figure No.	Name of Figure	Page No.
1	Fig. 3.1	System Architecture Diagram	23
2	Fig. 3.2	Data Flow Diagram	24
3	Fig. 3.3	ER Diagram	27
4	Fig. 3.4	UML class Diagram	28
5	Fig. 3.5	Use Case Diagram	29
6	Fig. 5.1	Fleet Dashboard	33
7	Fig. 5.2	Vehicle List	33
8	Fig. 5.3	Digital Twin	34
9	Fig. 5.4	OTA Campaign Manager	34
10	Fig. 5.5	Analytics & Health Scoring	35
11	Fig. 5.6	Alerts	35
12	Fig. 5.7	System Health	36
13	Fig. 5.8	Settings	36

LIST OF TABLES

S. No.	Table No.	Name of Figure	Page No.
1	Table 3.1	Technology Stack	16
2	Table 3.2	Rest API Endpoints	19
3	Table 3.3	Backend package summary	20
4	Table 3.4	Health score thresholds	20
5	Table 3.5	Vehicle profiles & operating ranges	21
6	Table 3.6	Docker Compose Services	22
7	Table 4.1	Software Requirement for project	30
8	Table 4.2	Software Requirement for runtime (Docker Container)	31
9	Table 4.3	Frontend Frameworks & Libraries	31
10	Table 4.4	Minimum Requirement for Hardware	32
11	Table 6.1	Known Limitations	38
12	Table 6.2	Major Project Roadmap	39

LIST OF ABBREVIATION

Abbreviation	Full form
API	Application Programming Interface
CORS	Cross-origin resource sharing
DTDL	Digital twin definition language
EDA	Event Driven Architecture
GPS	Global Positioning System
IEA	International Energy Agency
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MQTT	Message Queuing Telemetry Transport
OEM	Original Equipment Manufacturer
OTA	Over-the-Air
REST	Representational State Transform
SOC	State of Charge
SQL	Structured Query Language
TTL	Time to live
URL	Uniform resource locator
YAML	YAML Ain't Markup Language

ABSTRACT

Electric vehicles have witnessed rapid adoption worldwide, and fleets of hundreds of electric vehicles pose a significant challenge for fleet managers to monitor the vehicles, manage their batteries, and keep track of their connectivity status along with orchestrating software updates for the diverse range of vehicles. However, the current solutions lack a unified and vendor-independent perspective of the vehicles' status and operations.

The project proposes the development of a vendor-independent, event-driven Electric Vehicle Fleet Management Platform, named Axion. The proposed platform consumes 250 simulated electric vehicles' telemetry data using two different protocols, REST and MQTT. It normalizes the heterogeneous vendor data to a unified schema and maintains a real-time digital twin of the vehicles using the NoSQL data store Redis. A rule-based system calculates a real-time explainable health score (0 to 100) for the vehicles, classifying them as Healthy, Degraded, or Critical, considering the state of the batteries, temperature, and connectivity status of the vehicles. Axion also demonstrates the simulation of Over-the-Air (OTA) software update workflows, including the process of software rollout and rollbacks.

The backend uses Java 21 and Spring Boot 3.2, with Apache Kafka as the event backbone for fault-tolerant event handling. The backend uses a Python-based simulator to simulate 250 concurrent vehicles with five fault injection modes: normal driving, battery drain, temperature spike, network dropout, and OTA trigger. The frontend uses a dark-themed, premium React 18 dashboard with glassmorphism effects, animated KPI cards, health distribution charts, and real-time fleet monitoring on seven pages.

The entire system uses containerization with Docker Compose to deploy seven interconnected containers with health check-based startup sequence.

Keywords: Electric Vehicle, Fleet Management, Digital Twin, Telemetry, Apache Kafka, Event-Driven Architecture, OTA Updates, Health Scoring, Redis, Spring Boot, React, Docker

INDEX

SECTION	Page no.
List of Figures	1
List of Tables	2
List of Abbreviation	3
Abstract	4
Chapter 1 — Introduction	7-11
1.1 Background	7
1.2 Problem Statement	8
1.3 Objectives	9
1.4 Scope of Work	9
1.5 Organization of Report	10
Chapter 2 — Literature Survey	12-14
2.1 Existing EV Fleet Platforms	12
2.2 Digital Twin Technology	12
2.3 Event-Driven Architecture	13
2.4 OTA Update Mechanisms	13
Chapter 3 — Methodology	15-29
3.1 Overview of Methodology	15
3.2 Project Platforms used in Project	16
3.3 Proposed Methodology	17
3.4 Project Module	19
3.5 Diagrams	23
Chapter 4 — System Requirements	30-32
4.1 Software Requirements	30
4.2 Hardware Requirements	31
Chapter 5 — Results & Screenshots	33-36
Chapter 6 — Conclusion & Future Scope	37-39
6.1 Conclusion	37
6.2 Limitations	38

6.3 Future Scope	39
References	40

CHAPTER 1 - INTRODUCTION

This chapter will focus on the introduction to the Axion EV Fleet Management Platform, which is described as a vendor-neutral and event-driven platform for the ingestion, processing, and visualization of electric vehicle data in real-time. In this chapter, there will be an overview of the motivation behind the development of the platform, the background to electric vehicle fleet monitoring systems, the problems that are addressed, the objectives, the scope, and the organization of the report.

1.1 Background

The transportation industry has seen a major shift over the past few years with the rise of electric vehicles (EVs) over internal combustion engines. Various governments across the globe are promoting this shift to reduce carbon footprint. Various countries such as Norway, China, the Netherlands, and India are providing incentives to promote the use of EVs. The International Energy Agency (IEA) has announced that the sales of EVs worldwide exceeded 14 million units in 2023. This shows a high growth rate for EVs, which signifies the importance of this segment.

Organizations like logistics, ride-hailing services, and transportation services are adopting EVs. But this has brought about another set of problems to be addressed. The EVs have to be monitored with respect to factors like the charge left in the battery, the temperature of the battery, the consumption of power, etc. In addition to this, another major problem with EVs is that charging an EV takes a very long time compared to charging petrol or diesel cars. There are several problems that may come up during this process, like the vehicle may get overheated, or the utilization of power may not be proper, or the firmware may not be upgraded properly. There are several fleet management systems that are available today, but they have several limitations. For example, Tesla has come up with a Fleet API, which can be used to get access to the vehicle's telemetry. But this can only be used with Tesla cars. Similarly, cloud-based services like AWS IoT FleetWise can be used to process the vehicle's telemetry, but this requires a lot of cloud infrastructure to be set up.

These challenges, therefore, emphasize the importance of a vendor-independent and flexible system that can effectively obtain telemetry data from various car makers, transform the data into a standardized form, and intelligently monitor the conditions of the vehicles. This motivated the creation of the Axion EV Fleet Management Platform, a scalable system that can ingest, process, and visualize electric vehicle telemetry data in near real-time.

1.2 Problem Statement

However, there are various technical and operational challenges associated with the management of large electric vehicles, and the solutions provided currently do not fully handle these issues. According to the research and analysis of the electric vehicle fleet management system, the following key issues were identified:

1. Data Heterogeneity Across Manufacturers

It has been identified that different electric vehicle manufacturers use different formats for their telemetry data. For instance, the state of charge of the batteries of different vehicles may be represented as a value between 0 and 1, whereas for other vehicles, the same value may be represented as a percentage between 0 and 100. Similarly, the temperature value may be represented as Celsius or Fahrenheit, and the field names, such as soc_level, battery_percent, or state_of_charge, may represent the same value. This results in a challenge for the electric vehicles' fleet manager, as they have to manage vehicles of different manufacturers. Therefore, there is a need for a normalization layer to transform the different vendor-specific data formats into a common internal schema.

2. Lack of Real-Time Vehicle Health Assessment

Vehicle fleet operators are provided with real-time data; however, they are not aware of the vehicle's overall health. Fleet management should include an assessment of vehicle health. Vehicle operators should be aware of potential vehicle issues such as an increase in battery temperature or unusual energy consumption. A vehicle fleet management system should include a vehicle health score that reflects the vehicle's overall health and the factors affecting it.

3. Risks Associated with Over-the-Air (OTA) Software Updates

Today's electric vehicles use over-the-air software updates to enhance vehicle functionality and resolve software-related issues. However, initiating an over-the-air update without considering the vehicle's current state may lead to serious risks. For instance, an over-the-air update may be initiated when the vehicle's battery is low, the temperature is high, or network connectivity is unstable. In such a case, the over-the-air update may not be successful, leaving the vehicle in an unusable state.

4. Unreliable Network Connectivity

Electric vehicles are often in environments where network connectivity is poor, such as in tunnels, rural environments, or underground parking garages. During these times, the data cannot be sent to the backend system. The fleet management system should be designed to handle situations where the network is offline and data cannot be sent, and then seamlessly transition back to normal operation once network connectivity is restored.

5. Scalability and High Data Throughput

Fleet sizes for electric vehicles are typically large, and each of these vehicles produces a significant amount of data that needs to be sent to the system. If all the data is being sent

and then being processed in real-time, then the system will be overwhelmed and data will be lost. The system should be designed to handle high data throughput, and thus an event-driven system should be used, where data ingestion and data processing are decoupled.

1.3 Objectives

The main goals and objectives of this project are:

1. Design and implement a vendor-independent system for ingesting data, supporting both REST (HTTP) and MQTT-based data ingestion protocols for collecting data from the vehicles.

2. Design and implement a data normalization system based on the Adapter design pattern, enabling the standardization of data based on the vendor's data structure.

3. Design and implement a Digital Twin system that maintains the current state of each vehicle in real-time, ensuring that the data is not outdated.

4. Design and implement a system that scores the health of each vehicle based on parameters like state of charge, temperature, and connectivity, and generate a score between 0 and 100, along with an explanation of the score.

5. Design and implement an over-the-air (OTA) deployment system for the vehicles, considering safety parameters based on the health of the vehicles.

6. Design and implement a system that simulates the data sent by the vehicles, supporting up to 250 vehicles and multiple fault injection scenarios.

7. Design and implement an interactive system that enables monitoring of the health of the vehicles, key performance indicators, and real-time data using React.

8. Design and implement an end-to-end system that uses Docker Compose, enabling the deployment of the system.

1.4 Scope of Work

This project is about creating a full end-to-end prototype for an EV fleet management system. This will include all aspects of the data flow from telemetry generation to visualization for the operators. The system will comprise a backend section, a vehicle simulator, a frontend visualization section, and a containerized deployment system.

The backend part of this system will be implemented using Java 21 and Spring Boot 3.2. This part will handle API creation for telemetry ingestion and will be integrated with other tools such as Apache Kafka for event streaming, Redis for maintaining vehicle state (Digital Twin), and Eclipse Mosquitto for MQTT communication. The backend will comprise multiple sections for data ingestion, validation, health scoring, OTA update simulation, and telemetry processing.

For simulating real-world conditions, a vehicle simulator will be implemented using Python 3.12 with the asyncio library. This simulator will be able to generate telemetry

data for up to **250 vehicles simultaneously**, allowing the system to be tested under realistic fleet conditions. Different vehicle profiles and fault scenarios are used to simulate battery drain, temperature spikes, and other operational conditions.

The **frontend dashboard** is built using **React, TypeScript, Vite, and Tailwind CSS**, with additional libraries such as Recharts for visualization and Framer Motion for animations. The dashboard enables operators to monitor vehicle health, view fleet statistics, and observe real-time telemetry information through an interactive interface.

For deployment, the entire system is containerized using **Docker Compose**, which orchestrates multiple services including Kafka, Redis, Mosquitto, the backend API, the frontend dashboard, and the vehicle simulator. This setup allows the complete platform to be deployed and tested in a consistent and reproducible environment.

Certain features are outside the current scope of this project, including persistent database storage, user authentication and access control, WebSocket-based real-time communication, advanced machine learning analytics, and infrastructure monitoring tools. These features can be considered as potential improvements in future versions of the platform.

1.4 Organization of the Report

This report is divided into six chapters, with each chapter focusing on a unique aspect of the Axion EV Fleet Management Platform.

Chapter 1 is the introduction to the project, where the background of EV fleet management, the problem statement, the objectives of the project, and the scope of the project are discussed.

Chapter 2 is the literature survey, where the existing EV fleet management platforms, the concept of digital twin, event-driven architecture, and over-the-air updates are discussed.

Chapter 3 discusses the system design and architecture. In this chapter, the high-level system architecture, the telemetry data flow pipeline, the technology stack used in this project, and the API components are discussed.

Chapter 4 In this chapter, the implementation of the system modules, which include the Spring Boot backend, health scoring engine, OTA update simulation, vehicle simulator using Python, React dashboard, and Docker setup, are discussed.

Chapter 5 In this chapter, the results are presented, along with the demonstration of the system, including screenshots of the dashboard with the simulation running.

Chapter 6 concludes the report by summarizing the project outcomes, highlighting system limitations, and suggesting possible future improvements.

CHAPTER 2 – LITERATURE SURVEY

2.1 Existing EV Fleet Platforms

Several commercial solutions are available that offer fleet management solutions for electric vehicles. For example, the Fleet API provided by Tesla offers real-time vehicle status, location tracking, and command features. However, the Fleet API is only limited to Tesla vehicles and cannot be used for managing fleets with different manufacturers' vehicles. AWS IoT FleetWise is a cloud-based service for the ingestion of vehicle telemetry that can be used with different manufacturers of vehicles. However, it requires a high level of cloud infrastructure and expertise in the AWS environment.

Other fleet management solutions, such as Geotab and Samsara, offer various features such as vehicle tracking, telematics, and operation monitoring. Although these solutions offer high compatibility with various types of vehicles, the compatibility of these solutions with electric vehicle parameters such as battery health monitoring and thermal management is still limited. From the above analysis of the various solutions, it is clear that there is still a need for a self-hosted solution that is not limited to a specific manufacturer or vendor and can manage the telemetry of various manufacturers of vehicles in a unified manner..

2.2 Digital Twin Technology

Digital twin technology can be described as a process by which a virtual model of a real-world object or asset is created to receive continuous information from the real-world object. Digital twin technology was coined by Dr. Michael Grieves in 2003. Since then, the term has become quite common in the field of manufacturing, industrial IoT, smart infrastructure, and other related fields.

In fleet management systems, a digital twin represents the current state of a vehicle within the management system. Instead of asking for information from the vehicle whenever it is required, a digital twin model stores the current information such as the battery's state of charge, temperature, connectivity, and vehicle health.Large cloud providers offer digital twin services such as Microsoft Azure Digital Twins and AWS IoT TwinMaker, which allow organizations to build complex digital representations of industrial systems. However, these solutions are designed for large-scale industrial environments and require extensive cloud infrastructure.

In the case of the Axion platform, the concept of the digital twin is realized in a lightweight way using Redis as an in-memory data store. Each vehicle in the system has an associated digital twin that holds the latest data and health score. This enables the

system to access the data of the vehicles in an efficient way, and at the same time, removes the data automatically once the vehicles stop sending the data.

2.3 Event-Driven Architecture

Event-driven architecture (EDA) is a very popular architecture for designing modern distributed systems, especially for systems dealing with a large amount of data. In a typical system, the communication between different services occurs by the direct interaction of the services, but this results in a tight coupling between the systems, which might lead to a bottleneck. On the other hand, the EDA architecture enables the communication between different services by the publication of events to a message broker.

Apache Kafka is the most widely used platform for implementing the EDA architecture. This architecture enables the systems to handle a large amount of data efficiently, reliably, and scalably. This architecture stores the events as a distributed log, called a topic, and enables multiple systems to read the same data independently.In the Axion platform, Kafka acts as the **central event backbone** connecting telemetry ingestion with downstream processing components. Telemetry received through REST or MQTT is first normalized and then published to a Kafka topic. Processing services consume these events to update the vehicle's digital twin and compute its health score. This approach ensures that data ingestion and processing remain loosely coupled, improving scalability and system reliability.

The design also follows the canonical data model approach, whereby the data received from various vendors in the form of telemetry data is converted to a single standard format before it can be processed.

2.4 OTA Update Mechanisms

Over the Air (OTA) updates enable the remote update of software, firmware, and configurations by the car manufacturer, eliminating the need for the car to visit a service center for the purpose of the update. Tesla Motors pioneered the use of OTA updates for electric vehicles. This technology enables the improvement of the capabilities of the car, its battery, and its safety features by remote software update.

Although there are many benefits to using OTA updates, there are several risks associated with the process. There is a risk of failure of the update process if the car's battery level is low, the system's operating temperature is high, or the network connectivity is unstable.

To mitigate these risks, the following best practices are used in the latest OTA deployment systems. Health-gated deployment ensures that the software updates are deployed only after the vehicle is in a state that satisfies the conditions, such as the level of charge in the vehicle's battery and the temperature. Canary deployments involve testing the software updates on a small group of vehicles before rolling out the updates to

the entire fleet. There is also the option of rolling back the software version in case the updates fail or unexpected problems occur.

The Axion platform utilizes the above-mentioned best practices in the OTA simulation module, which checks the health of the vehicle before the software updates. Though the Axion platform simulates the firmware updates, the architecture is similar to that used in the latest EV software update systems.

CHAPTER 3 — METHODOLOGY

3.1 Overview of Methodology

The development of the Axion platform followed an iterative, module-first approach. Rather than building the entire system in a single pass, we divided the platform into independent, loosely coupled modules — a backend ingestion and processing engine, a vehicle simulator, and a frontend dashboard — and developed each one incrementally while integrating them through a central event backbone.

The core methodology is rooted in Event-Driven Architecture (EDA). In a traditional request-response system, each component directly calls the next, creating tight coupling and making the system fragile under load. In an event-driven system, components communicate by publishing and consuming events through a message broker. This means the telemetry ingestion module does not need to know anything about the health scoring engine or the digital twin service — it simply publishes a normalised telemetry event to Apache Kafka, and any number of downstream consumers can pick it up independently. This architectural choice was made to ensure scalability, fault tolerance, and the ability to add new features without modifying existing code.

We also adopted the Digital Twin paradigm for vehicle state management. Instead of querying each vehicle directly for its current status, the platform maintains a lightweight in-memory replica (digital twin) of every vehicle in Redis. This twin is updated every time new telemetry arrives and automatically expires if the vehicle stops sending data, providing a natural mechanism for detecting offline vehicles.

The development lifecycle followed these phases:

1. Requirements Analysis — Studied existing EV fleet platforms, identified gaps, and defined project objectives.

2. Architecture Design — Designed the event-driven pipeline, defined the canonical data model, and chose the technology stack.

3. Module Development — Built the backend, simulator, and frontend as independent modules.

4. Integration & Testing — Connected all modules through Kafka and Docker Compose, and validated end-to-end data flow with 250 simulated vehicles.

5. Deployment— Containerised the entire stack for one-command deployment

3.2 Project Platforms Used in Project

The Axion platform uses a carefully chosen set of open-source technologies, each selected for its maturity, community support, and suitability for the specific problem it addresses.

Table 3.1 Technology Stack

LAYER	TECHNOLOGY	VERSION
Backend	Java , Spring Boot ,Spring WebFlux	Java 21, Spring 3.2
MQTT Broker	Eclipse Mosquitto	2.0
State Store	Redis	7.0 Alpine
Messaging	Apache Kafka + Zookeeper	Confluent 7.5
Frontend	React , TypeScript , Vite,Tailwind CSS	React 18 , Vite 6.3
Charts	Recharts	2.15
Animations	Framer Motion	Latest
Simulator	Python, asyncio , YAML config	Python 3.12
Deployment	Docker, Docker Compose	7 containers

Why these choices:Java 21 + Spring Boot 3.2 was chosen for the backend because Spring Boot provides production-ready Kafka integration, Redis auto-configuration, and REST controller scaffolding with minimal boilerplate. Java 21 brings virtual threads and modern language features.

Apache Kafka was chosen over alternatives like RabbitMQ or Redis Pub/Sub because Kafka provides durable, partitioned, replayable event logs — meaning events are not lost even if a consumer is temporarily offline.

Redis was chosen as the state store because digital twins require sub-millisecond read latency and automatic expiry (TTL) for stale data detection — both of which Redis provides natively.**Python 3.12 with asyncio** was chosen for the simulator because Python's asyncio allows running 250 concurrent vehicle coroutines in a single process with minimal overhead.

React 18 with TypeScript was chosen for the frontend for its component-based architecture, strong typing, and rich ecosystem of visualisation and animation libraries.

Docker Compose was chosen for deployment to ensure anyone can run the entire platform with a single command, regardless of their operating system or installed dependencies.

3.3 Proposed Methodology

The Axion platform follows a layered, event-driven architecture where data flows through a well-defined pipeline from generation to visualisation. The system is divided into three independent tiers: a Python-based simulator (data source), a Java-based backend (processing engine), and a React-based frontend (operator interface). These tiers communicate through Apache Kafka as the central event backbone and Redis as the shared state store

System Architecture:

The architecture consists of the following layers:

1. **Data Generation Layer (Python Simulator)** Generates realistic telemetry payloads for 250 concurrent electric vehicles. Each vehicle runs as an independent asyncio coroutine, producing data every 100 milliseconds based on its assigned profile (sedan, truck, or sport) and scenario (normal drive, battery drain, temperature spike, network dropout, or OTA trigger).

2. **Ingestion Layer (Spring Boot REST + MQTT):*** Receives raw telemetry via dual protocols — HTTP POST (REST API) and MQTT publish. The ingestion controller uses the Adapter design pattern to convert vendor-specific JSON payloads into a standard CanonicalTelemetryEnvelope format, regardless of how the vendor names its fields. The TelemetryValidator then checks that all critical fields are present and within acceptable bounds.

3. **Event Streaming Layer (Apache Kafka):** Valid, normalised telemetry events are published to Kafka's `telemetry.normal` topic. OTA-related events flow through a separate `ota.events` topic. Kafka decouples the ingestion path from the processing path — the ingestion service can accept data at full speed even if downstream consumers are temporarily processing slowly.

4. **Processing Layer (Kafka Consumers):** The TelemetryConsumer reads events from Kafka and triggers two actions for each event: (a) updating the vehicle's digital twin state in Redis, and (b) computing the vehicle's health score through the rule-based health scoring engine.

5. **State Management Layer (Redis Digital Twins):** Each vehicle's current state is stored as a serialised JSON object in Redis under the key pattern `digital_twin:{vehicleId}`. Every twin carries a 120-second TTL — if a vehicle stops sending telemetry, its twin automatically expires, naturally marking the vehicle as absent from the fleet.

6. **Presentation Layer (React Dashboard):** The frontend polls the backend REST API every 3–5 seconds and renders the fleet state through eight fully functional pages with animated KPI cards, health distribution charts, and real-time vehicle monitoring.

Data Flow Pipeline:

The telemetry data flows through the system in the following sequence:

1. **Generation:** The Python simulator creates telemetry payloads containing battery SOC percentage, speed, battery temperature, motor temperature, ambient temperature, and odometer reading.

2. **Transmission:** Payloads are sent to the backend via HTTP POST or MQTT publish.

3. **Ingestion & Normalization:**The Spring Boot ingestion controller applies the Adapter pattern to convert vendor-specific formats into the CanonicalTelemetryEnvelope.

4. **Validation:** TelemetryValidator checks that all critical fields are present and within bounds.

5. **Event Publishing:** Valid telemetry is published to Kafka's `telemetry.normal` topic.

6. **Consumption & Processing:** TelemetryConsumer reads from Kafka, updates the digital twin, and computes the health score.

7. **State Storage:** The updated digital twin is stored in Redis with a 120-second TTL. Timestamp-based ordering ensures stale data never overwrites newer data.

8. **API Serving:** The Fleet REST API reads digital twin states from Redis and serves them to the React dashboard.

Table 3.2 : REST API Endpoints:

METHOD	ENDPOINT	DESCRIPTION
POST	/api/v1/telemetry	Ingest raw telemetry payload
GET	/api/v1/fleet/summary	Fleet KPIs(toal , online , health dist , events/s)
GET	/api/v1/fleet/vehicles	All vehicles with latest telemetry
GET	/api/v1/fleet/vehicle/{id}	Single vehicle full digital twin
POST	/api/v1/ota/trigger	Trigger OTA update for a vehicle

3.4 Project Modules

The Axion platform is composed of six distinct modules, each handling a specific concern:

Module 1: Backend Ingestion & Processing Engine (Java/Spring Boot)

The backend is structured as a clean, layered Spring Boot application with 47 Java source files organized into nine logical packages

When a telemetry payload arrives via REST, TelemetryIngestionService uses RestTelemetryAdapter to transform vendor-specific JSON into the canonical format. After validation, the event is published to Kafka. On the consumer side, TelemetryConsumer reads from Kafka, calls DigitalTwinService.update() which checks timestamp-based freshness, runs the health scoring engine, and writes the updated twin to Redis with a 120-second TTL.

Table 3.3: backend Package Summary:

Package	Purpose	Key Classes
adapter	Vendor data normalization	RestTelemetryAdapter, TelemetryAdapter
api	REST controllers	FleetController, VehicleController
config	Infrastructure configuration	KafkaConsumerConfig, KafkaProducerConfig, MqttConfig, RedisConfig, WebConfig
consumer	Kafka event processing	TelemetryConsumer
health	Health scoring engine	HealthScoreEngine, HealthRules, HealthScoreResult
model	Core domain models	CanonicalTelemetryEnvelope, DigitalTwinState, TelemetrySnapshot
service	Business logic	DigitalTwinService, OtaService, TelemetryIngestionService, ThroughputTracker
validation	Payload validation	TelemetryValidator
ota	OTA orchestration	OtaCampaign, OtaJob, OtaOrchestratorService

Module 2: Health Scoring Engine

The health scoring system uses a rule-based approach where every score is fully explainable. The engine starts every vehicle at a base score of 100 and deducts points based on predefined thresholds.

Table 3.4: Health Score Threshold:

Factor	Warning Threshold	Penalty	Critical Threshold	Penalty
Battery SOC	< 30%	−15 pts	< 15%	−30 pts
Battery Temperature	> 45°C	−15 pts	> 55°C	−30 pts
Connectivity	—	—	Offline	−30 pts

The final score is classified as: HEALTHY (80–100), DEGRADED (50–79), or CRITICAL (0–49). Each deduction generates a human-readable explanation string stored alongside the score in the digital twin.

Module 3: OTA Simulation Module

When an operator triggers an OTA update, the OtaService first checks vehicle eligibility — battery SOC must be above minimum threshold, battery temperature below critical level, and vehicle not in CRITICAL health state. If passed, an OTA job is created and published to the `ota.events` Kafka topic, progressing through the state machine: PENDING → IN_PROGRESS → SUCCESS / FAILURE (simulated 80% success rate). The campaign orchestrator supports canary-style phased rollouts and automatic rollback on excessive failures

Module 4: Python Vehicle Simulator

The simulator uses asyncio to run 250 vehicles concurrently. Vehicle profiles define operating ranges.

Table 3.5: Vehicle Profiles and Operating ranges:

Profile	Temperature Range	SOC Range	Speed Range
Sedan (Default)	25–32°C	80–95%	40–80 km/h
Truck	35–40°C	85–100%	40–60 km/h
Sport	30–38°C	70–95%	60–110 km/h

Five fault injection scenarios (Normal Drive, Battery Drain, Temp Spike, Network Dropout, OTA Trigger) test different aspects of the system. Out of 250 vehicles, 20 are manually configured and 230 are auto-generated across three fleet groups.

Module 5: React Frontend Dashboard

The dashboard uses a dark colour scheme with glassmorphism effects and animated KPI cards. It comprises eight pages:

Fleet Dashboard , Vehicle List , Vehicle Detail , OTA manager , Analytics , Alerts, System Health , Setting page.

Module 6: Docker Compose Deployment

The entire platform is containerised using Docker Compose with seven interconnected containers:

Table 3.6: Docker Compose services:

Container	Image	Port	Purpose
axion-zookeeper	Confluent 7.5	2181	Kafka coordination
axion-kafka	Confluent 7.5	9092	Event streaming
axion-redis	Redis 7.0 Alpine	6379	Digital twin storage
axion-mosquitto	Eclipse Mosquitto 2.0	1883	MQTT broker
axion-backend	Spring Boot (custom)	8080	Backend services
axion-frontend	React + Nginx (custom)	80	Dashboard UI
axion-simulator	Python 3.12 (custom)	—	Telemetry simulator

Every container includes health checks, and startup ordering is enforced via depends_on with service_healthy conditions.

3.5 Diagram

Fig.3.1 System Architecture Diagram

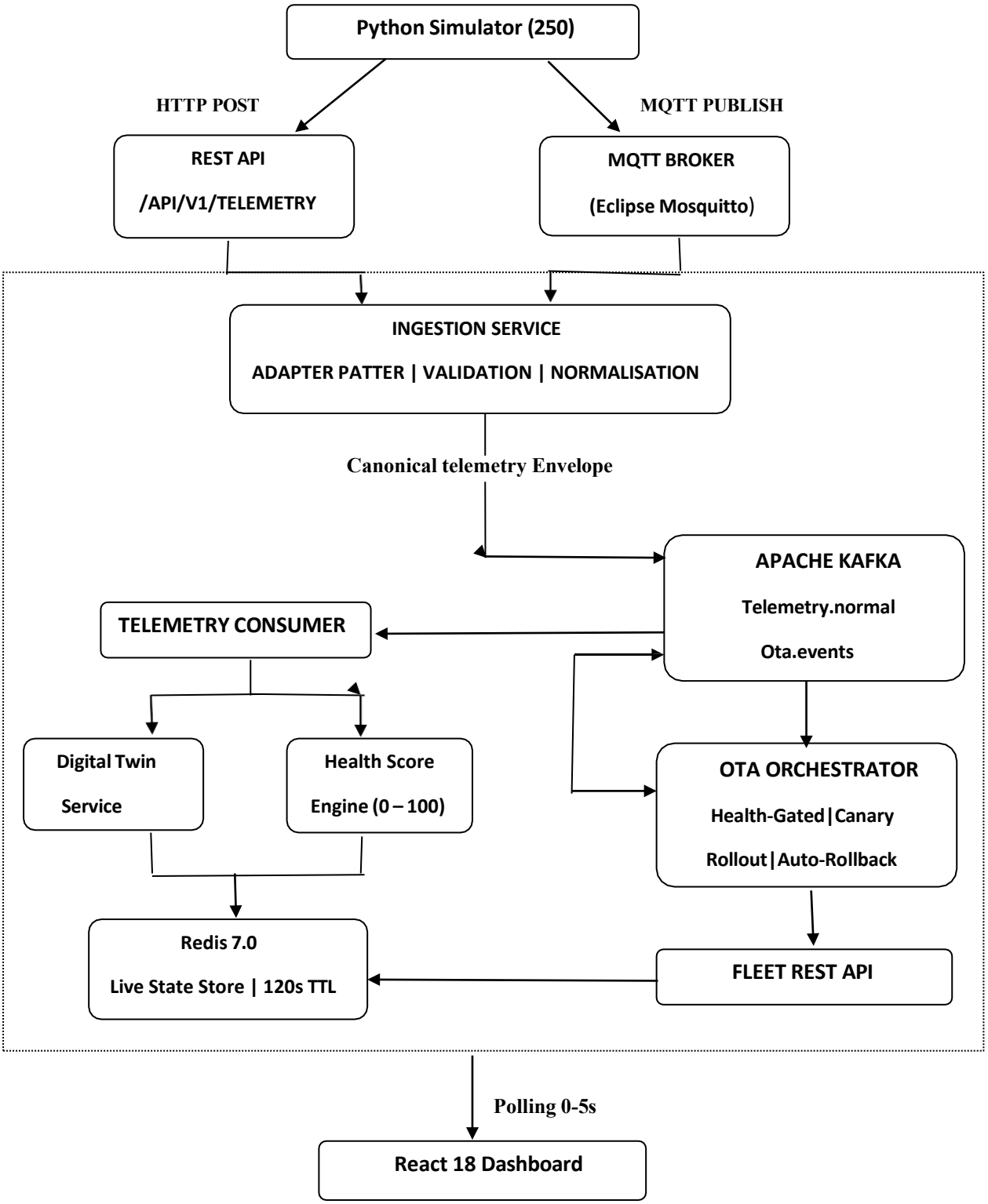
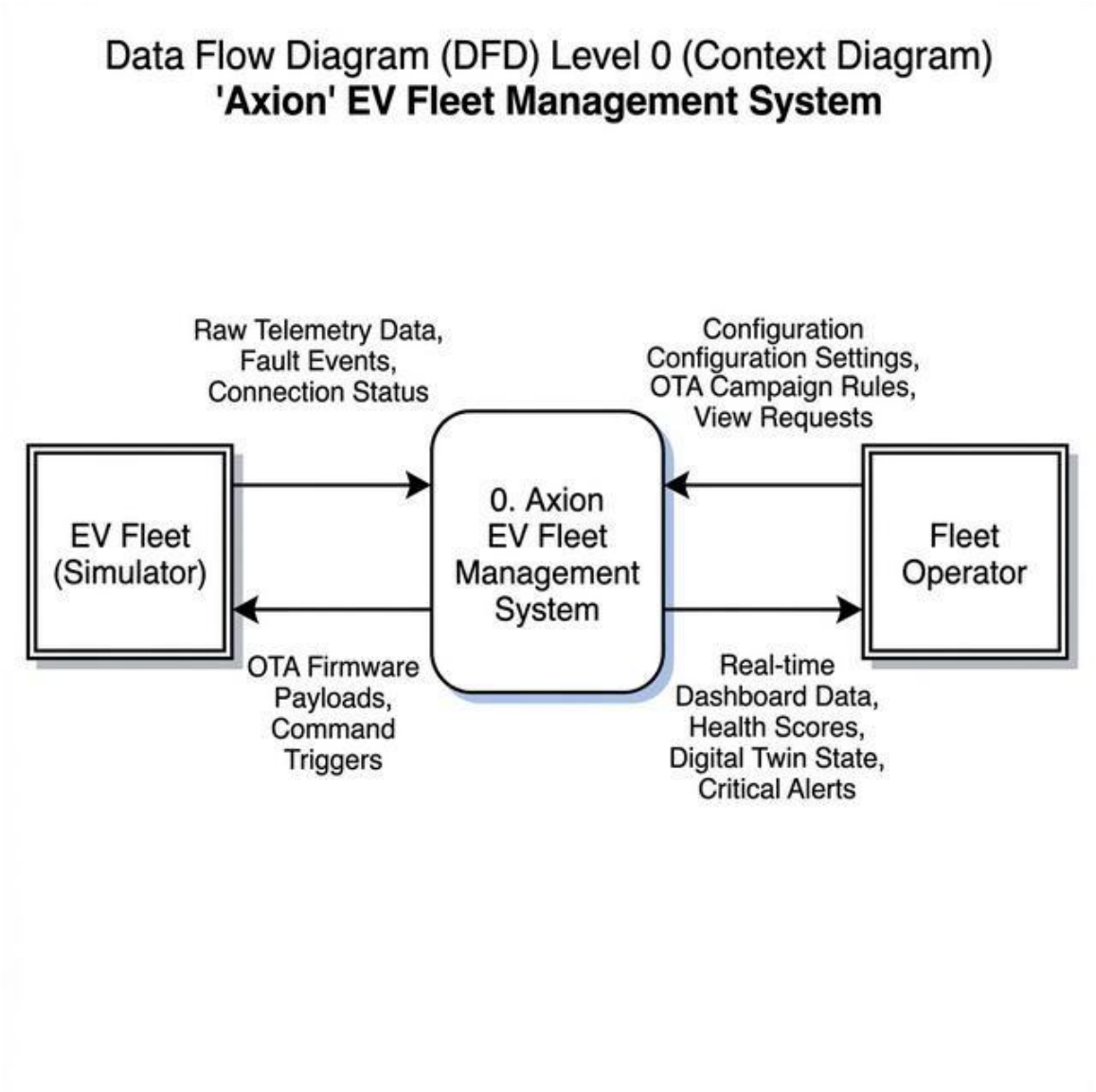
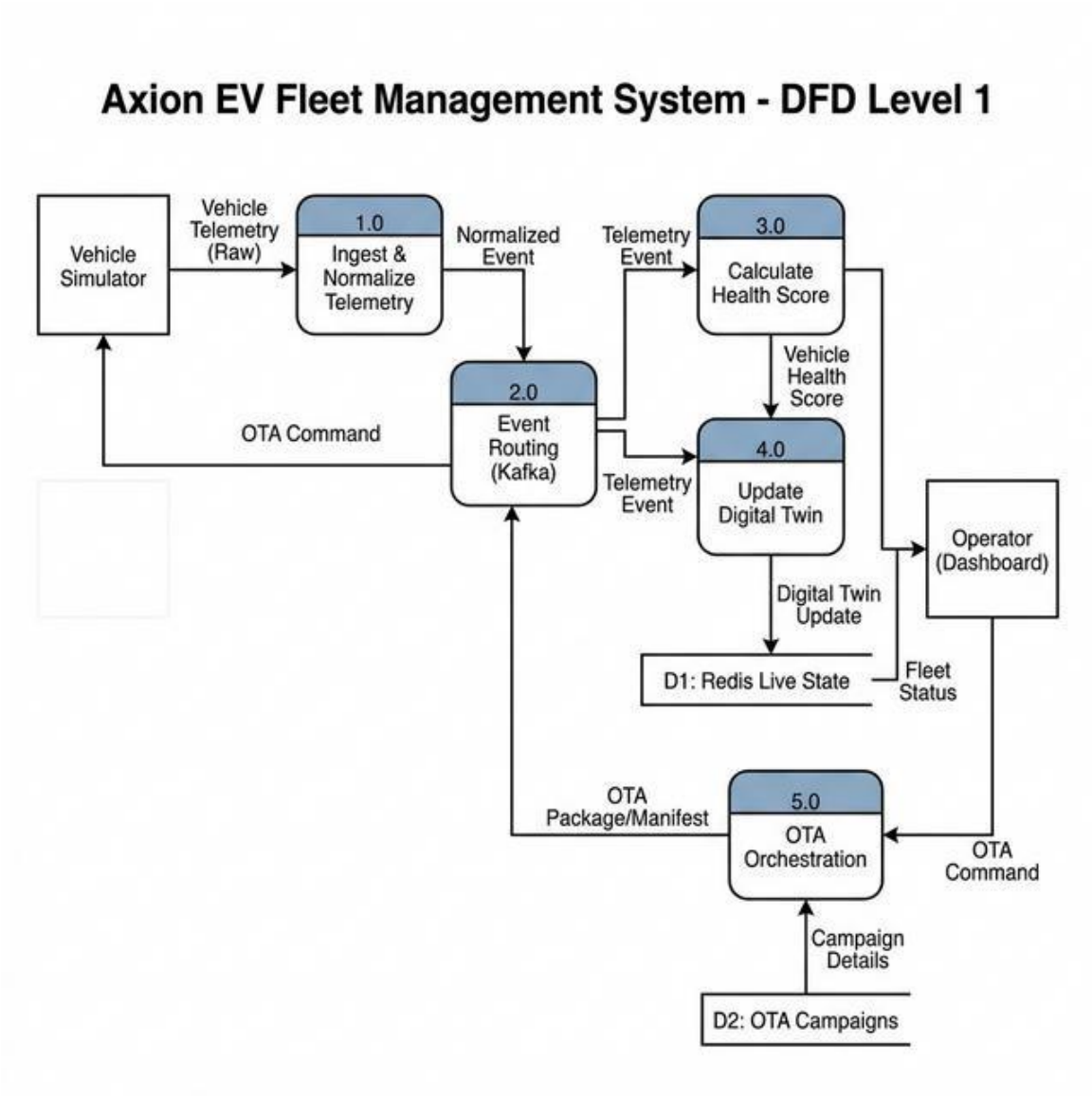


Fig.3.2 Data Flow Diagram

Level 0 :



Level 1:


Axion EV Fleet Management System - DFD Level 1
Vehicle Simulator
Vehicle Telemetry (Raw)
1.0
Ingest & Normalize Telemetry
Normalized Event
Telemetry Event
3.0
Calculate Health Score
Vehicle Health Score
2.0
Event Routing (Kafka)
OTA Command
4.0
Update Digital Twin
Telemetry Event
Operator (Dashboard)
Digital Twin Update
D1: Redis Live State
Fleet Status
OTA Package/Manifest
5.0
OTA Orchestration
OTA Command
Campaign Details
D2: OTA Campaigns

Level 2:

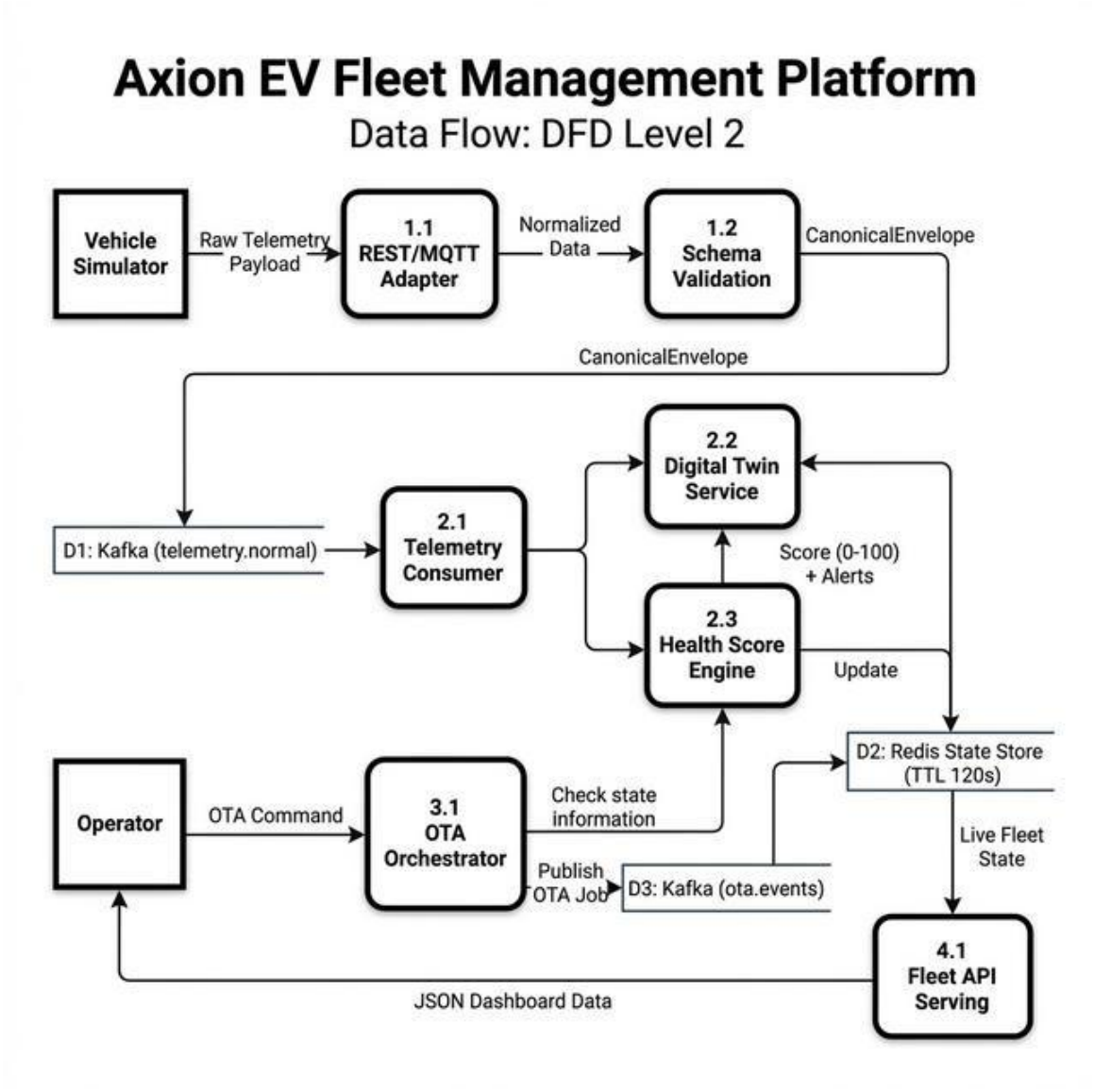

Axion EV Fleet Management Platform
Data Flow: DFD Level 2
Vehicle Simulator
Raw Telemetry Payload
1.1 REST/MQTT Adapter
Normalized Data
1.2 Schema Validation
CanonicalEnvelope
CanonicalEnvelope
2.2 Digital Twin Service
D1: Kafka (telemetry.normal)
2.1 Telemetry Consumer
Score (0-100) + Alerts
2.3 Health Score Engine
Update
D2: Redis State Store (TTL 120s)
Operator
OTA Command
3.1 OTA Orchestrator
Check state information
Publish OTA Job
D3: Kafka (ota.events)
Live Fleet State
4.1 Fleet API Serving
JSON Dashboard Data

Fig: 3.3: Entity Relationship Diagram

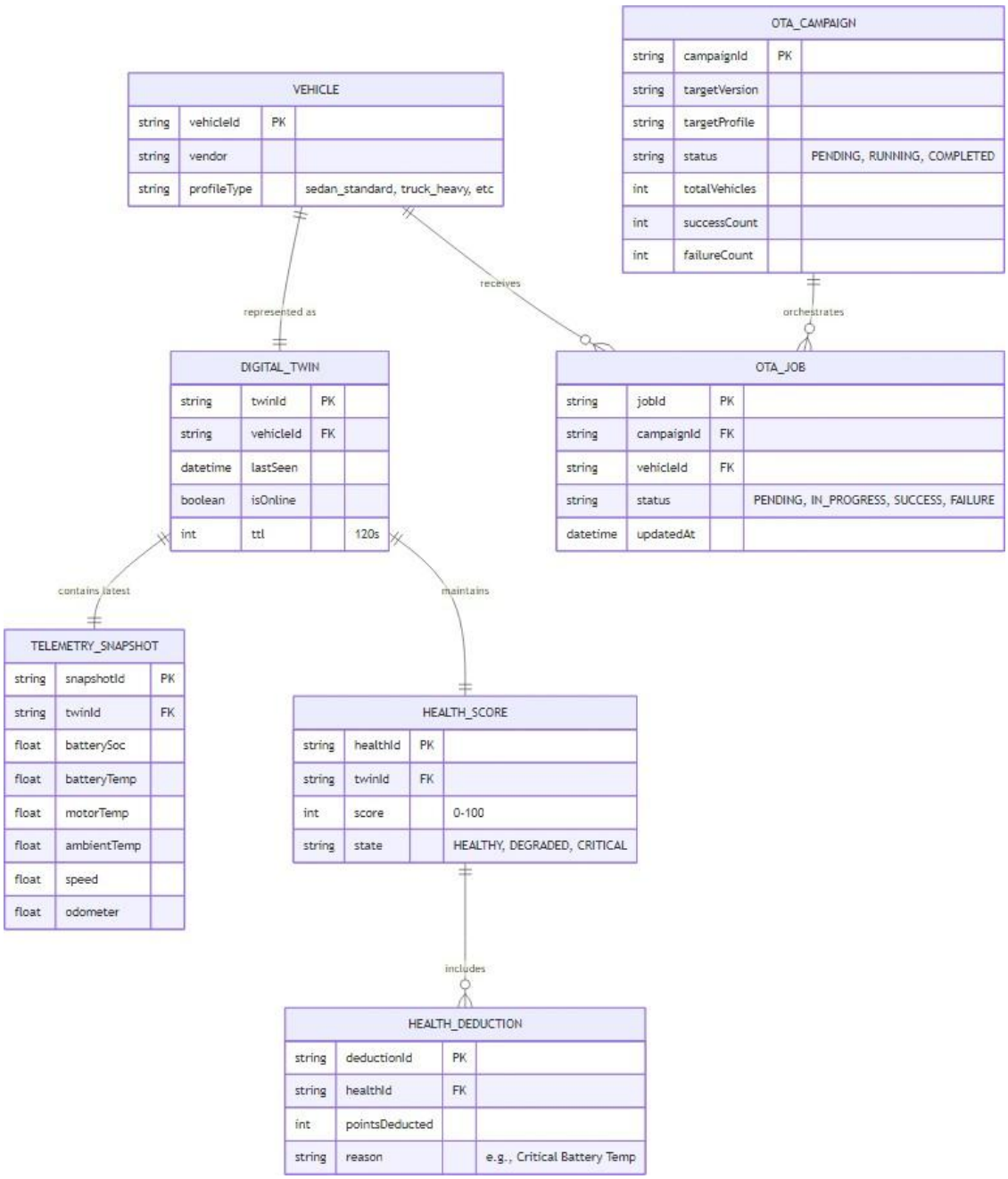


Fig.3.4: UML Class Diagram

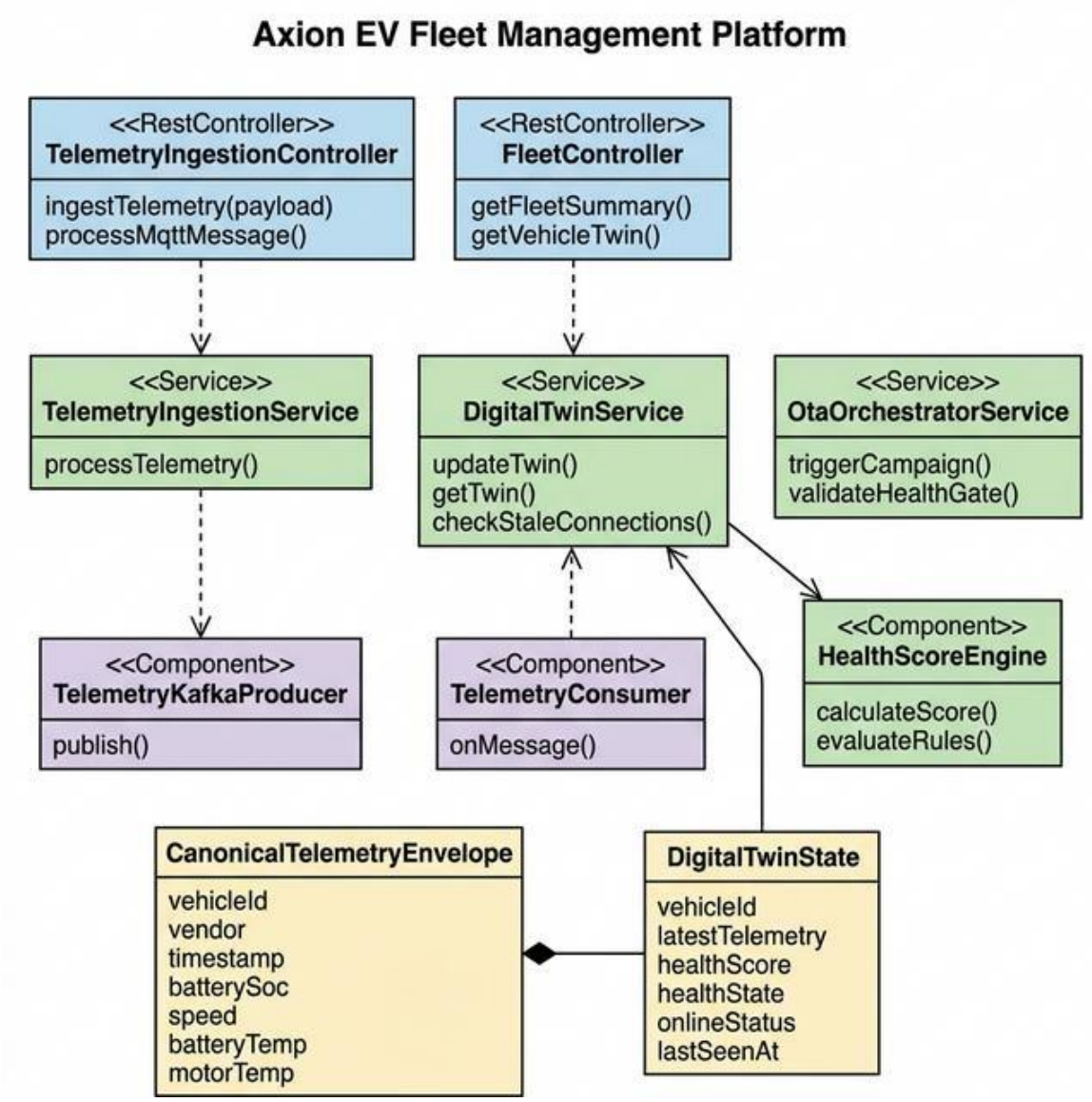
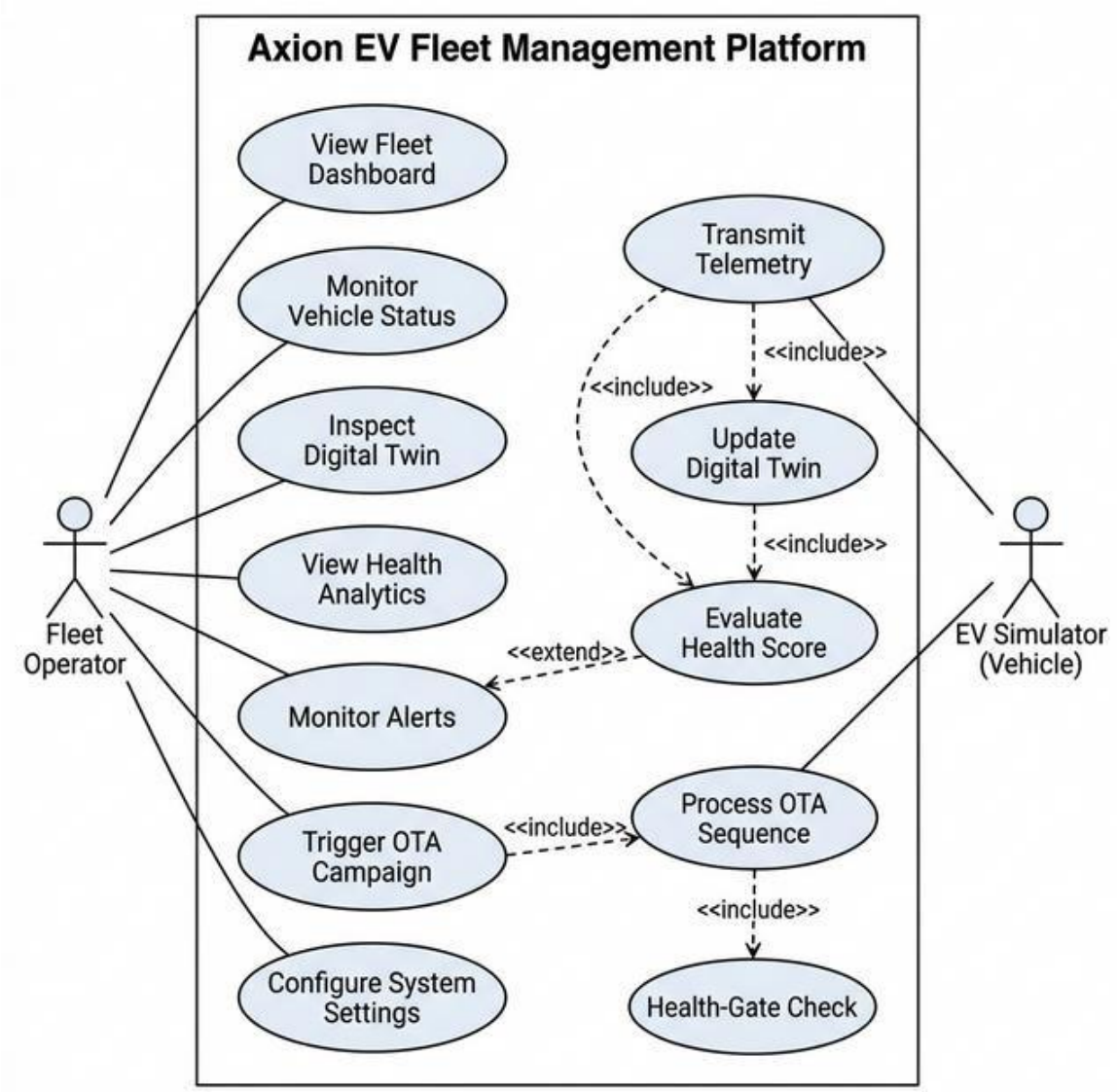


Fig.3.5: Use Case Diagram



CHAPTER 4 – SYSTEM REQUIREMENTS

This chapter outlines the software and hardware requirements necessary to develop, run, and test the Axion EV Fleet Management Platform. Since Axion is a fully containerised application deployed using Docker Compose, the runtime requirements are minimal — any machine with Docker installed can run the entire platform regardless of operating system.

4.1 Software Requirements

The following software tools, frameworks, and runtime environments are required for the development and execution of the Axion platform.

Table 4.1: Software Requirements for Development

S.NO.	SOFTWARE	VERSION	PURPOSE
1	Java Development Kit (JDK)	21 (LTS)	Backend application development and compilation
2	Apache Maven	3.9+	Build automation and dependency management for Java
3	Spring Boot Framework	3.2.x	Backend REST API, Kafka, Redis, and MQTT integration
4	Node.js	18+ (LTS)	Frontend build toolchain and package management
5	npm (Node Package Manager)	9+	Installing and managing frontend dependencies
6	Python	3.12	Vehicle simulator development and execution
7	Docker Desktop	24+	Container runtime for building and running images
8	Docker Compose	v2.20+	Multi-container orchestration and deployment
9	Git	2.40+	Version control and source code management
10	Visual Studio Code / IntelliJ IDEA	Latest	Integrated development environment (IDE)

Table 4.2: Software Requirements for Runtime (Docker Containers)

S. No.	Software	Version / Image	Container Name	Purpose
1	Apache Kafka	Confluent 7.5	axion-kafka	Event streaming and message brokering
2	Apache Zookeeper	Confluent 7.5	axion-zookeeper	Kafka cluster coordination and management
3	Redis	7.0 Alpine	axion-redis	In-memory digital twin state storage
4	Eclipse Mosquitto	2.0	axion-mosquitto	MQTT broker for IoT telemetry protocol
5	Nginx	Alpine (latest)	axion-frontend	Static file serving for React dashboard
6	OpenJDK	21-slim	axion-backend	Java runtime for Spring Boot application
7	Python	3.12-slim	axion-simulator	Runtime for 250-vehicle simulator

Table 4.3: Frontend Libraries and Frameworks

S. No.	Library / Framework	Version	Purpose
1	React	18.x	Component-based UI development
2	TypeScript	5.x	Type-safe JavaScript for maintainable code
3	Vite	6.3	Fast build tool and development server
4	Tailwind CSS	3.x	Utility-first CSS framework for styling
5	Shadcn UI	Latest	Accessible, customisable component primitives
6	Recharts	2.15	Data visualisation and charting library
7	Framer Motion	Latest	Animation library for smooth UI transitions
8	React Router	6.x	Client-side routing and navigation

Operating System Compatibility:

Since the entire Axion platform runs inside Docker containers, it is operating system agnostic. The platform has been developed and tested on the following operating systems:

- Windows 11 (with Docker Desktop and WSL 2)

- Ubuntu 22.04 LTS (with Docker Engine)

- macOS Ventura (with Docker Desktop)

Browser Compatibility:

The React frontend dashboard is compatible with all modern web browsers:

- Google Chrome 100+

- Mozilla Firefox 100+

- Microsoft Edge 100+

- Safari 16+

4.2 Hardware Requirements

The following tables specify the minimum and recommended hardware configurations for running the Axion platform.

Table 4.4: Minimum Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th Gen) or AMD Ryzen 5 2600
RAM	8 GB DDR4
Storage	20 GB free disk space (for Docker images and containers)
Network	Stable internet connection for pulling Docker images
Display	1366 × 768 resolution (for dashboard viewing)
Operating System	Windows 10/11, Ubuntu 20.04+, or macOS 12+

CHAPTER 5 – RESULTS & SCREENSHOTS

Fig.5.1: Fleet Dashboard

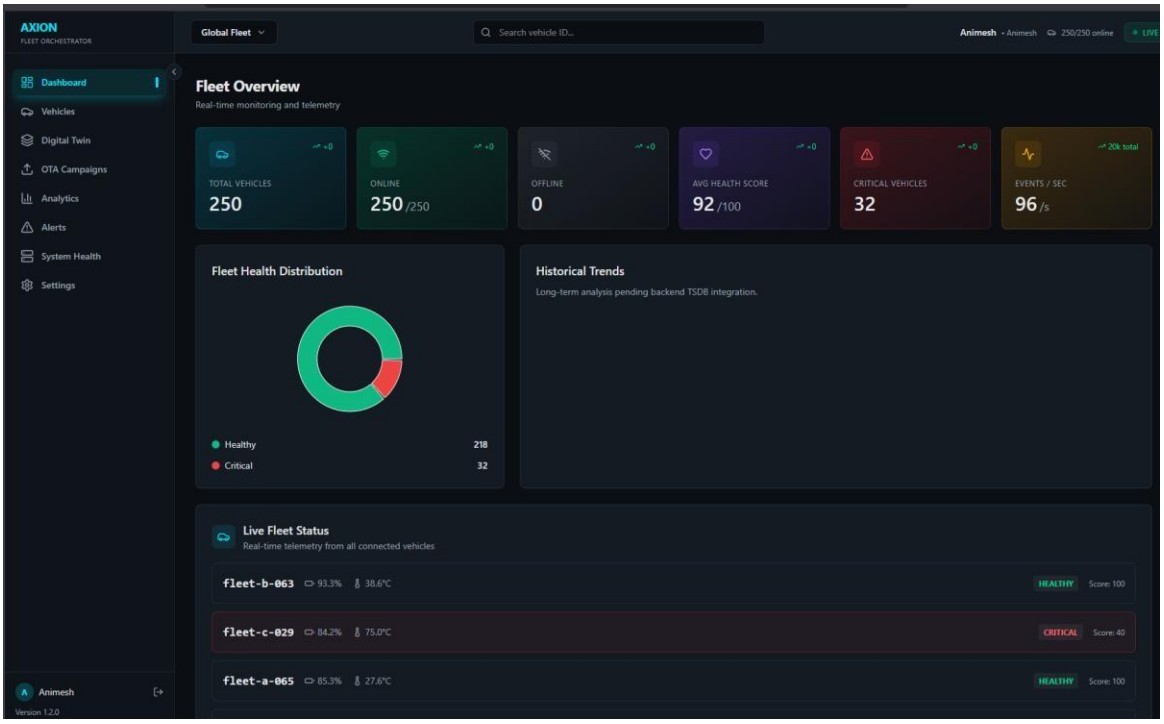


Fig.5.2: Vehicle List

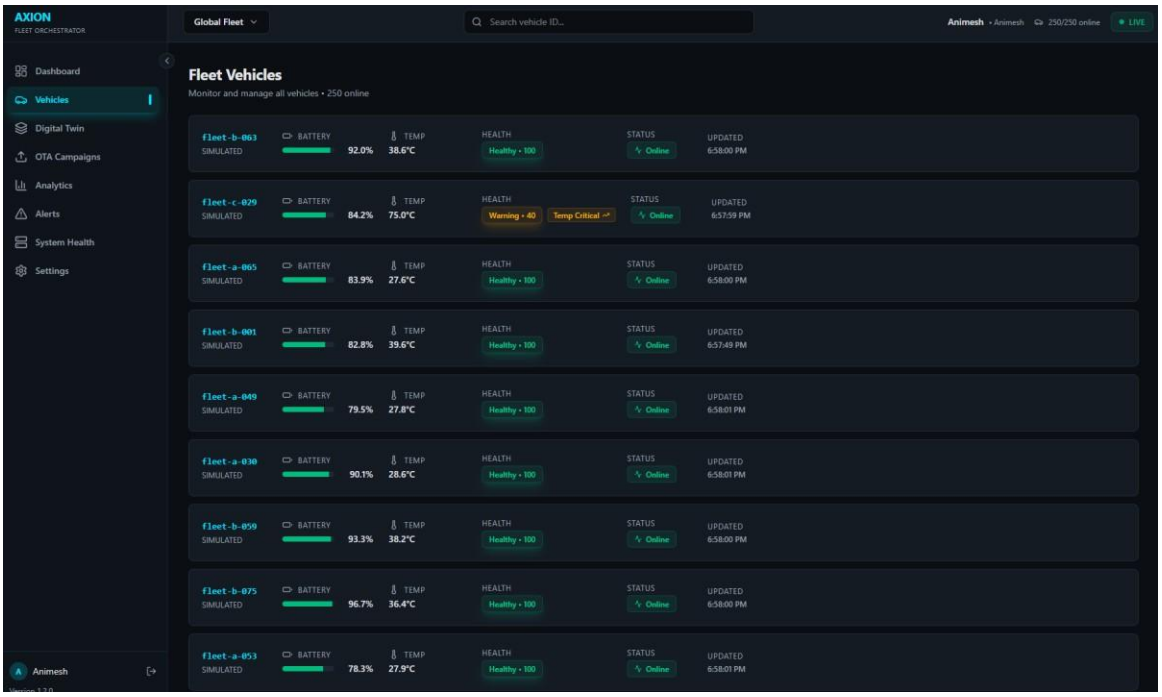


Fig. 5.3: Digital Twin

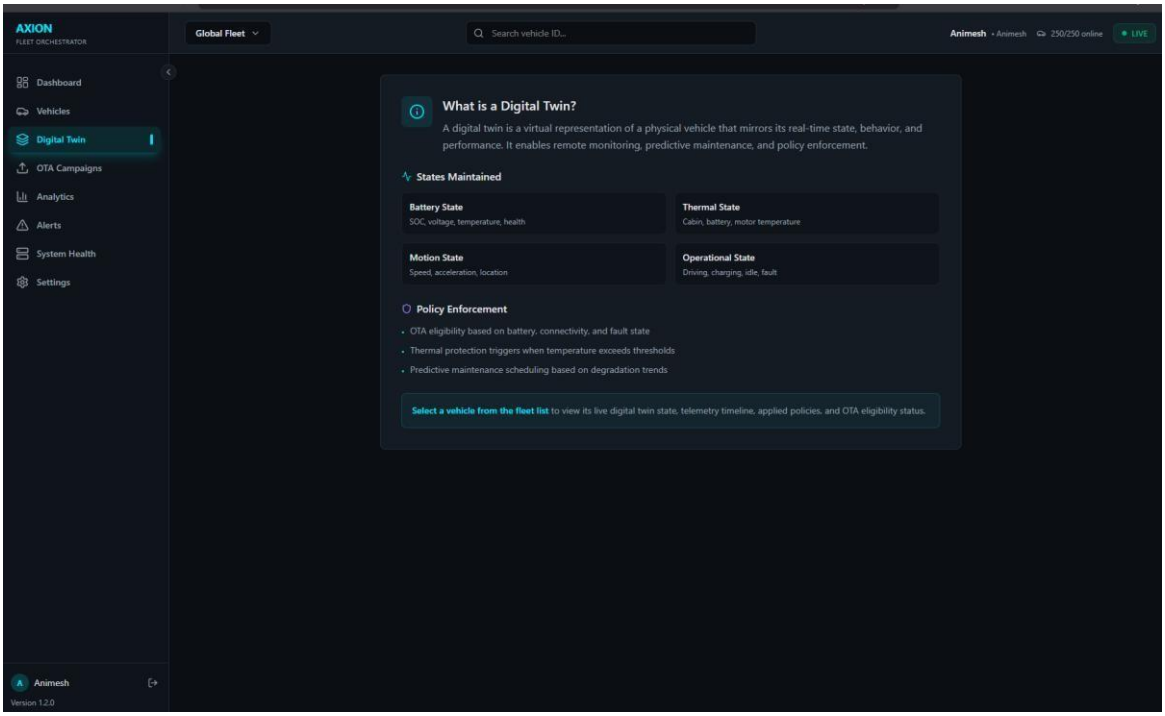


Fig.5.4 : OTA Campaign Manager

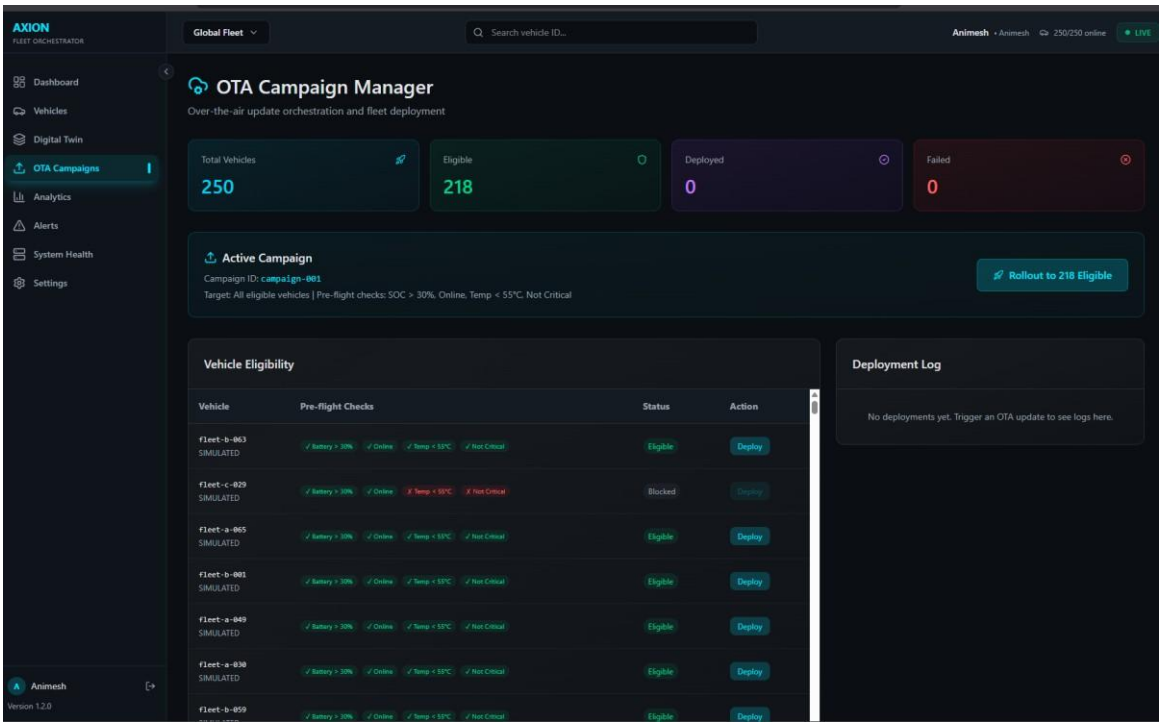


Fig.5.5 : Analytics & Health Scoring

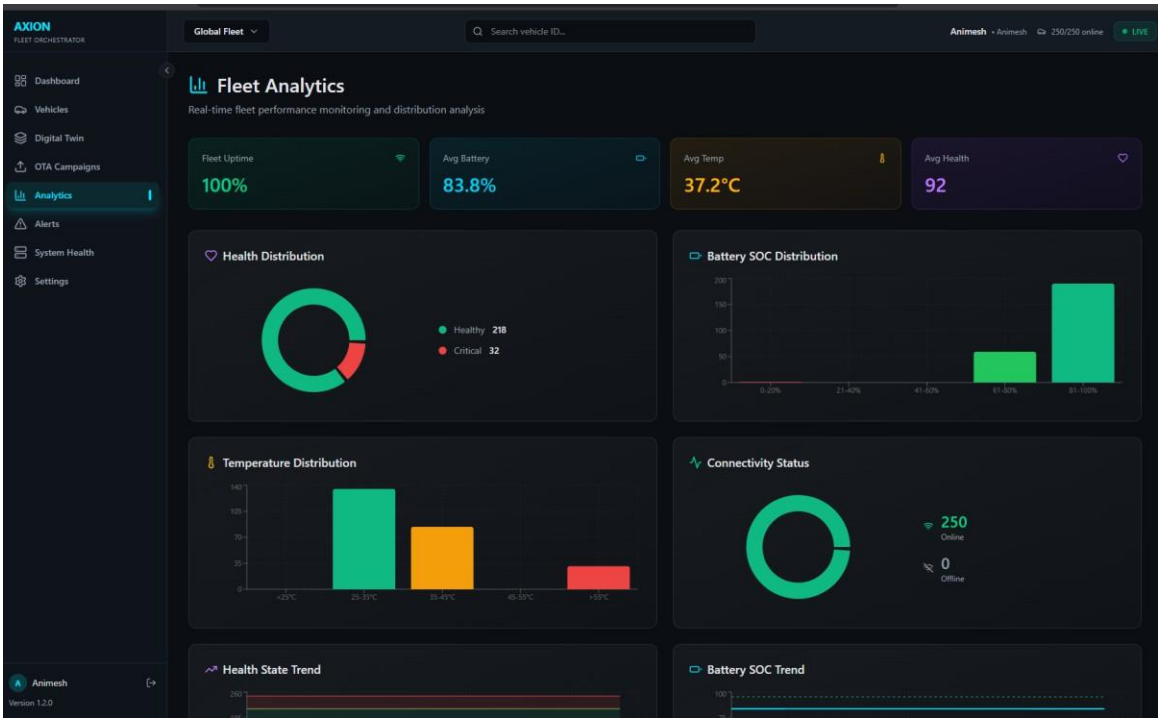


Fig.5.6: Alerts

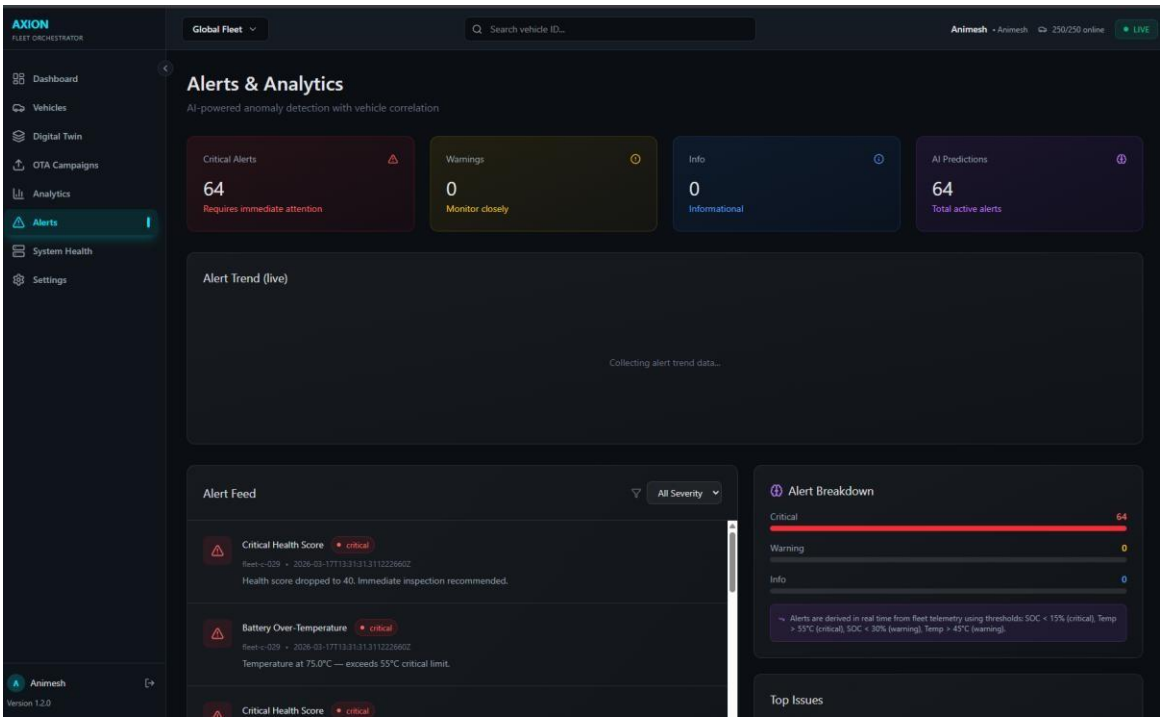


Fig.5.7 : System Health

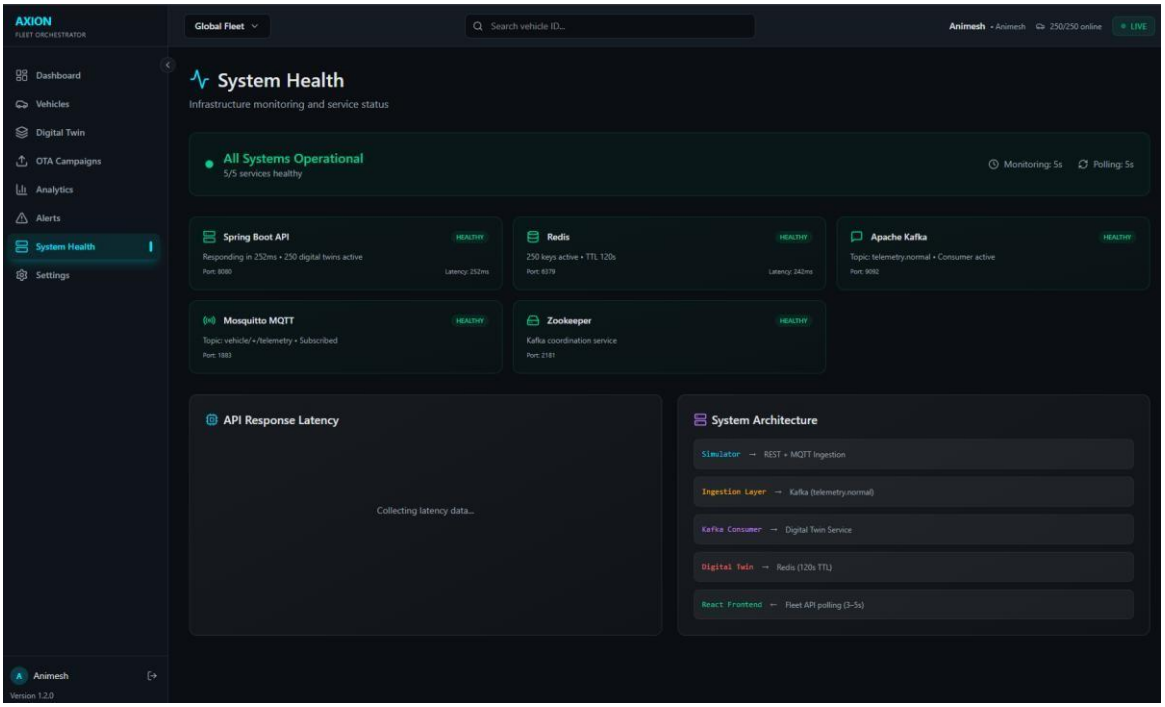
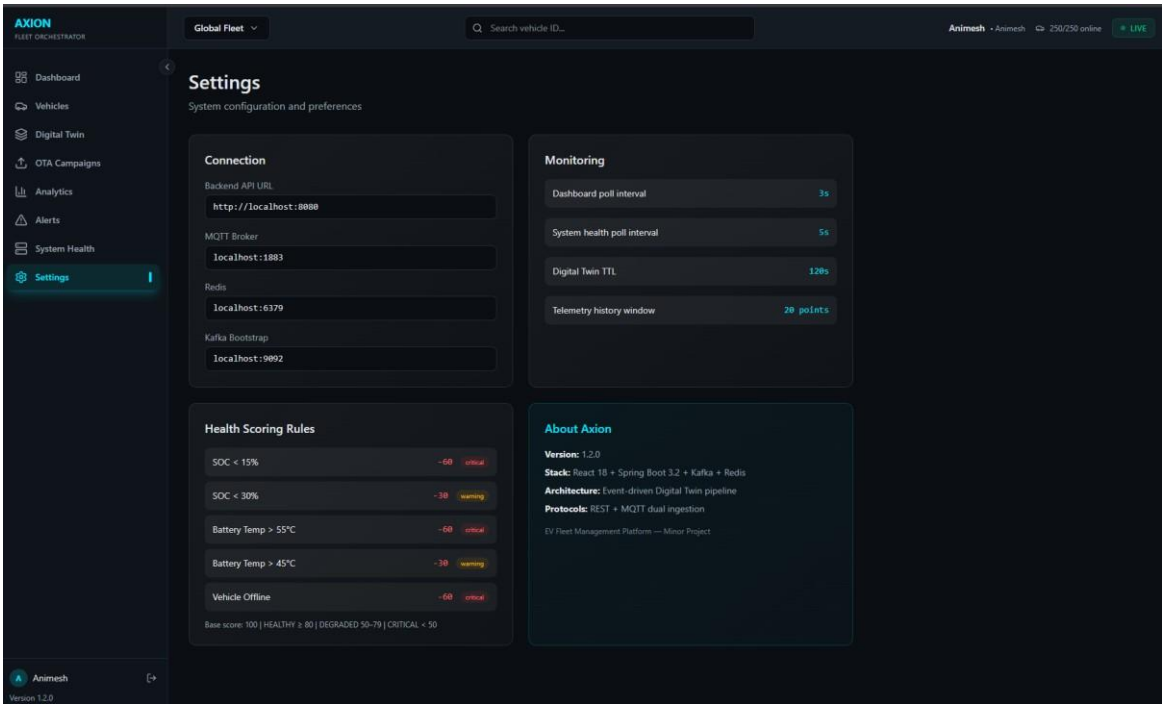


Fig.5.8: Settings



CHAPTER 6 — CONCLUSION & FUTURE SCOPE

6.1 Conclusion

- This Minor Project has led to the development of a fully functional, vendor-independent EV Fleet Management Platform named Axion. The platform was designed and developed from an empty repository using an event-driven architecture that is capable of ingesting, normalizing, and processing the telemetry of 250 simulated electric vehicles in real time.
- The main technological achievements of the Minor Project include:
- Design and implementation of a real-time dual-protocol telemetry ingestion pipeline (REST and MQTT) capable of ingesting heterogeneous vendor telemetry using the Adapter pattern.
- Design and implementation of a real-time Digital Twin Engine using Redis as the backend store, enabling fast access to the latest state of all vehicles.
- Design and implementation of an Explainable Health Scoring Engine that assesses the state of the vehicles and generates human-readable explanations for the health of each vehicle.
- A Python-based simulator with the ability to produce telemetry for 250 concurrent vehicles, including realistic fault injection.
- Development of a **modern React-based dashboard** with multiple pages that allow operators to monitor fleet status, vehicle health, and system alerts.
- Deployment of the complete system using **Docker Compose**, enabling one-command startup of all required services with proper container health checks.

Experience has been gained in the following areas during the development of the Axion platform: event-driven systems, digital twin systems, real-time processing, message brokers, containerized application deployment, and more.

6.2 Limitations

Although the system demonstrates the core functionality of an EV fleet management platform, several limitations remain in the current implementation.

Table 6.1 Known Limitations

Limitation	Impact
No persistent database	All vehicle data is lost when Redis TTL (120 seconds) expires
No user authentication	API endpoints are open and lack access control
No historical data	Only last-known-state snapshots available
No machine learning	System reacts to current conditions but cannot predict failures
Polling-only frontend	3–5s polling intervals, not truly real-time
No observability	No Prometheus, Grafana, or structured logging

6.3 Future Scope (Major Project)

The Minor Project establishes the **architectural foundation of the Axion platform**. In the upcoming Major Project phase, the system will be extended to include advanced capabilities and production-grade infrastructure.

Table 6.2: Major Project Roadmap

Phase	Feature	Timeline
Phase 1	TimescaleDB + PostgreSQL for storing 30-day telemetry history	Weeks 1–3
Phase 2	JWT authentication and role-based access control	Weeks 3–4
Phase 3	WebSocket communication to replace polling with real-time updates	Weeks 4–5
Phase 4	Machine learning-based predictive analytics	Weeks 5–8
Phase 5	Advanced OTA deployment with campaign management and rollback	Weeks 6–8
Phase 6	Root cause analysis Timelin	Weeks 7–8
Phase 7	Observability & Monitoring	Weeks 8–9
Phase 8	Final system optimization, documentation, and load testing	Weeks 9–10

The **core API contracts and canonical data schema** developed in the Minor Project will remain unchanged, ensuring backward compatibility while new capabilities are added during the Major Project phase.

REFERENCES

1] Tesla, Inc., "Tesla Fleet API Documentation," 2024. [Online]. Available: https://developer.tesla.com/docs/fleet-api

[2] Amazon Web Services, "AWS IoT FleetWise Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/iot-fleetwise/

[3] Geotab Inc., "Geotab EV Fleet Management Solutions," 2024. [Online]. Available: https://www.geotab.com/fleet-management-solutions/electric-vehicles/

[4] M. Grieves, "Digital Twin: Manufacturing Excellence through Virtual Factory Replication," whitepaper, Florida Institute of Technology, 2014.

[5] Redis Ltd., "Redis Documentation — Data Types and TTL," 2024. [Online]. Available: https://redis.io/docs/

[6] Apache Software Foundation, "Apache Kafka Documentation," 2024. [Online]. Available: https://kafka.apache.org/documentation/

[7] G. Hohpe and B. Woolf, "Enterprise Integration Patterns," Addison-Wesley, 2003.

[8] F. Lambert, "Tesla pushes new OTA software update with dozens of improvements," Electrek, 2023. [Online]. Available: https://electrek.co/

[9] A. Taivalsaari and T. Mikkonen, "On the Development of IoT Systems," IEEE International Conference on Software Architecture, 2018.

[10] Spring Framework, "Spring Boot 3.2 Reference Documentation," 2024. [Online]. Available: https://docs.spring.io/spring-boot/